What is an operator ?
---
operator is a symbol that performs a specific task.

syntax
---
operand operator operand
a + b

MySQL Operators and Their Usage

What are Operators?

Operators are special symbols or keywords used in MySQL to perform operations on data. They help in calculations, comparing values, filtering records, combining conditions, manipulating strings, and performing bitwise operations.

Types of Operators

1. **Arithmetic Operators**
2. **Comparison Operators**
3. **Assignment Operators**
4. **Logical Operators**
5. **String Operators**
6. **Bitwise Operators**
7. **NULL Operators**
8. **Special Operators**

---

1. Arithmetic Operators

Definition

Arithmetic operators are used to perform mathematical calculations on numeric values.

Operator	Description	Example	Result

`+`	**Addition**	`10 + 5`	**15**
`-`	**Subtraction**	`10 - 5`	**5**
`*`	**Multiplication**	`10 * 5`	**50**
`/`	**Division**	`10 / 5`	**2**
`%`	**Modulus (Remainder)**	`10 % 3`	**1**

STEP 1: Create Table

```
CREATE TABLE Products (
 id INT,
 name VARCHAR(100),
 price DECIMAL(10,2),
 qty INT
 );
```

STEP 2: Insert Records

```
INSERT INTO Products VALUES
 (1, 'Pen', 10.00, 100),
 (2, 'Notebook', 25.00, 50),
 (3, 'Pencil', 5.00, 200),
 (4, 'Eraser', 3.00, 300),
 (5, 'Marker', 15.00, 80),
 (6, 'Scale', 20.00, 150),
 (7, 'Sharpener', 8.00, 120),
 (8, 'Geometry Box', 120.00, 40),
 (9, 'Sketch Pen', 60.00, 75),
 (10, 'Crayons', 90.00, 55);
```

SELECT Queries

Q1. Calculate the price after adding 12% GST.

```
SELECT name, price, price + (price * 0.12) AS price_with_tax
 FROM Products;
```

Q2. Calculate the discounted price after reducing ₹5.

```
SELECT name, price, price - 5 AS discounted_price
 FROM Products;
```

Q3. Find the total inventory value of each product.

```
SELECT name, price, qty, price * qty AS total_inventory_value
 FROM Products;
```

Q4. Calculate the cost of purchasing 10 units.

```
SELECT name, price, price * 10 AS price_for_10_units
 FROM Products;
```

Q5. Display products whose quantity is not divisible by 50.

```
SELECT *
 FROM Products
 WHERE qty % 50 <> 0;
```

UPDATE Queries

Q1. Add 2 units to every product.

```
UPDATE Products
 SET qty = qty + 2;
```

Q2. Reduce the price of every product by ₹5.

```
UPDATE Products
 SET price = price - 5;
```

Q3. Double the quantity of Eraser.

```
UPDATE Products
 SET qty = qty * 2
 WHERE name = 'Eraser';
```

Q4. Reduce the quantity of Marker to half.

```
UPDATE Products
 SET qty = qty / 2
 WHERE name = 'Marker';
```

Q5. Increase the price of Notebook by ₹20.

```
UPDATE Products
 SET price = price + 20
 WHERE name = 'Notebook';
```

DELETE Queries

Q1. Delete products whose quantity is less than 100.

```
DELETE FROM Products
 WHERE qty < 100;
```

Comparison Operators in MySQL

What are Comparison Operators?

Comparison Operators are used to compare two values or expressions. They return TRUE (1) or FALSE (0) based on whether the comparison condition is satisfied.

These operators are mainly used with the WHERE clause to filter records.

Types of Comparison Operators

Operator	Description	Example
`=`	Equal to	`salary = 50000`
`!=` or `<>`	Not Equal to	`department != 'HR'`
`>`	Greater Than	`salary > 50000`
`<`	Less Than	`age < 25`
`>=`	Greater Than or Equal To	`age >= 18`
`<=`	Less Than or Equal To	`salary <= 40000`

Note: In MySQL, both `!=` and `<>` represent Not Equal To.

STEP 1: Create Table

```
CREATE TABLE StudentScores (
 id INT,
 name VARCHAR(100),
 subject VARCHAR(50),
 marks INT
 );
```

STEP 2: Insert Records

```
INSERT INTO StudentScores VALUES
 (1, 'Alice', 'Math', 95),
 (2, 'Bob', 'Math', 76),
 (3, 'Charlie', 'Math', 59),
 (4, 'Diana', 'Math', 88),
 (5, 'Eve', 'Math', 40),
 (6, 'Frank', 'Math', 95),
 (7, 'Grace', 'Science', 67),
 (8, 'Henry', 'Science', 82),
 (9, 'Ivy', 'English', 91),
 (10, 'Jack', 'English', 55),
 (11, 'Kevin', 'Math', 73),
 (12, 'Lily', 'Science', 48),
 (13, 'Mia', 'English', 100),
 (14, 'Noah', 'Math', 64),
 (15, 'Olivia', 'Science', 79);
```

SELECT Queries

Q1. Display students who scored more than 80.

```
SELECT *
 FROM StudentScores
 WHERE marks > 80;
```

Q2. Display students who scored 60 or more.

```
SELECT *
 FROM StudentScores
 WHERE marks >= 60;
```

Q3. Display students who scored less than 50.

```
SELECT *
 FROM StudentScores
 WHERE marks < 50;
```

Q4. Display students who scored exactly 95.

```
SELECT *
 FROM StudentScores
 WHERE marks = 95;
```

Q5. Display students who did not score 95.

```
SELECT *
 FROM StudentScores
 WHERE marks != 95;
```

UPDATE Queries

Q1. Increase marks by 10 for students scoring below 50.

```
UPDATE StudentScores
 SET marks = marks + 10
 WHERE marks < 50;
```

Q2. Increase marks by 5 for students scoring 60 or below.

```
UPDATE StudentScores
 SET marks = marks + 5
 WHERE marks <= 60;
```

Q3. Add 2 bonus marks to students scoring above 90.

```
UPDATE StudentScores
 SET marks = marks + 2
 WHERE marks > 90;
```

Q4. Add 3 bonus marks to students scoring 80 or more.

```
UPDATE StudentScores
 SET marks = marks + 3
 WHERE marks >= 80;
```

Q5. Reduce 1 mark for students who did not score exactly 95.

```
UPDATE StudentScores
 SET marks = marks - 1
 WHERE marks != 95;
```

DELETE Queries

Q1. Delete students who scored below 50.

```
DELETE FROM StudentScores
 WHERE marks < 50;
```

Q2. Delete students who scored 60 or below.

```
DELETE FROM StudentScores
 WHERE marks <= 60;
```

Q3. Delete students who scored above 90.

```
DELETE FROM StudentScores
 WHERE marks > 90;
```

Q4. Delete students who scored 80 or more.

```
DELETE FROM StudentScores
 WHERE marks >= 80;
```

Q5. Delete students whose marks are not equal to 95.

```
DELETE FROM StudentScores
 WHERE marks != 95;
```